AutoSquad

AutoSquad - Fabián Andrés Souto
2nd DAM - Academic Year 2025/2026

Table of Contents

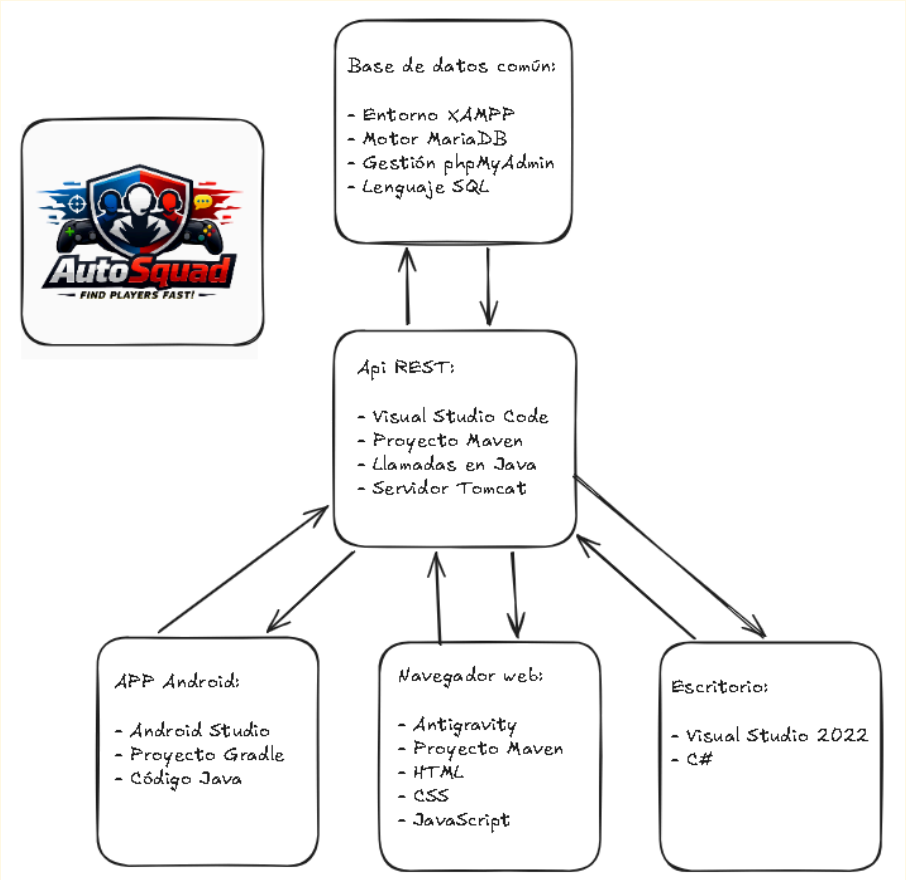
- System Architecture
- API
- Database
- Web Application
- Mobile Application
- Desktop Application
- Security
- Deployment
- User & Maintenance Guide
- Future Improvements

1. System Architecture

The goal of this application is to facilitate communication between players who want to play in teams but need additional members as quickly as possible. AutoSquad is composed of three connected applications that communicate through a REST API, which in turn interacts with the database. Everything works together so that each user can find their ideal teammate as soon as possible.

Main Technologies:

- API: Java
- Database: MySQL
- Mobile App: Android (Java)
- Web App: HTML, CSS, JavaScript
- Desktop App: C#



2. API

Endpoint	Method	Description
/login	POST	Log in
/register	POST	Register user
/caja/create	POST	Create chat rooms
/caja/list	GET	List chat rooms
/mensaje/create	POST	Send messages
/mensaje/list	GET	Get messages

Authentication:

- Currently, authentication is based on username and password. Example:

```
String correo = req.getParameter("correo");
String contra = req.getParameter("contra");

PreparedStatement ps = conn.prepareStatement(
        "SELECT id_usuario, nombre, contra FROM USUARIO WHERE correo=?");

ps.setString(1, correo);
ResultSet rs = ps.executeQuery();

if (rs.next() && contra.equals(rs.getString("contra"))) {

    out.print("{\"success\":true," +
            "\"id\":" + rs.getInt("id_usuario") + "," +
            "\"nombre\":\"" + rs.getString("nombre") + "\"}");
} else {
    out.print("{\"success\":false,\"error\":\"Credenciales incorrectas\"}");
}
```

That part of my API returns a JSON indicating whether it's correct or incorrect, along with the data:

- Successful login:

```
{
  "success": true,
  "id": 1,
  "nombre": "Fabian"
}
```

- Failed login:

```
{
  "success": false,
  "error": "Credenciales incorrectas"
}
```

3. Database

Tables:

- Usuario (data of different users):

id_usuario	nombre	correo	contra	modos	rango

  - id_usuario: unique ID
  - nombre: account name
  - correo: login/contact method
  - contra: password
  - modos: (future implementation) user specialties
  - rango: (future implementation) user statistics

- Caja (each created chat room):

id_caja	id_creador	id_juego	visibilidad	requisitos	jugadores_buscados	hora_creacion

  - id_caja: box ID
  - id_creador: creator ID
  - id_juego: game for which players are being searched
  - visibilidad: (future implementation) indicates whether it will be sent based on JSON
  - requisitos: user requirements
  - jugadores_buscados: number of players needed
  - hora_creacion: creation time (used to determine lifespan)

- Mensaje (data of each sent message):

id_mensaje	id_usuario	id_caja	texto	fecha_hora

  - id_mensaje: unique message ID
  - id_usuario: user who sends it
  - id_caja: related chat room
  - texto: message content
  - fecha_hora: timestamp (used for ordering)

- Juego (currently supported games):

id_juego	nombre
1	Valorant
2	LoL
3	CS2

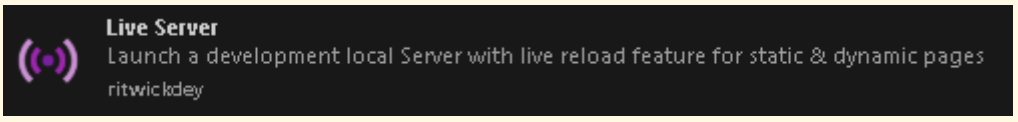
- id_juego: supported game ID
- nombre: game name

Important Rules:

- Each box must exist for one hour (after that it will be hidden but never deleted)
- Users already chatting can continue after that hour (connection is not interrupted)

4. Web Application

- Development tool: Antigravity
- Languages: HTML, CSS, JavaScript
- Structure: Each HTML file contains its own CSS and JavaScript. In the future, these will be separated, but keeping them together avoids common issues and confusion.
- The project uses Maven and can be easily tested with the “Live Server extension”:



- The client application consumes the API using HTTP requests with “fetch”, sending and receiving JSON data:

```
const response = await fetch(`${API_BASE}/login`, {
    method: "POST",
    body: formData
});
```

5. Mobile Application

- Development tool: Android Studio
- Language: Java
- Structure: Activities, Models, and Adapters
- The client application consumes the API via HTTP requests, sending and receiving JSON data

6. Desktop Application

- Development tool: Visual Studio 2022
- Project: Windows Forms (.NET Framework)
- Language: C#
- Structure: Forms, Models, and Services
- The client (Form) calls an ApiService, using GET/POST requests to retrieve JSON data, which is then transformed into messages, chat boxes, and other required elements

7. Security

Currently, AutoSquad uses basic authentication with username and password, and session handling through local variables. There is no use of tokens or password encryption, which introduces certain risks. However, improvements are planned for early versions, as this is currently a prototype to test user reception. Both the API and applications perform basic validation of user input to ensure proper data formatting and prevent errors.

8. Deployment

- Backend hosted locally (localhost:8080)

- Android app available for emulator or mobile (possible Play Store release)
- Web app accessible via browser (future indexing planned)
- Desktop app downloadable from AutoSquad.com
- Development environment: Local machine
- Future production: Hosting service or self-hosted server with redundancy

9. User & Maintenance Guide

1. Create the database in XAMPP (use “ServidorTablas.txt”) and start Apache and MySQL
2. Start Tomcat with the API (.war file in “autosquad-api”)
3. Use Live Server to run the web app (“autosquad-web”) or deploy to a free hosting service like glitch.com
4. Mobile app runs directly once installed (developed in Android Studio)
5. Desktop app runs directly after opening (developed in Visual Studio 2022)

10. Future Improvements

- Code optimization (e.g., reduce and separate CSS files)
- Add more games like Fortnite, Clash Royale, Overwatch 2, and Minecraft
- Implement a payment system for cosmetics (profile customization, colored text, animated profile pictures)
- Add AI moderation for harmful messages
- Age restrictions and verification for certain games

- Improve password encryption and implement two-factor or biometric authentication
- Ability to create permanent or long-lasting groups for content creators
- Move from local IP to a dedicated company IP for proper indexing and system consistency